Network and IoT Security | NIoSec

Summary

CONTENTS

1. Introduction ... 3
2. TLS ... 3
2.1. Specification ... 3
2.1.1. Record protocol ... 4
2.1.2. Subprotocols ... 4
2.1.3. Handshake ... 4
2.2. Modern TLS Variants ... 4
2.2.1. DTLS ... 4
2.2.2. QUIC ... 5
2.3. Attacks ... 5
2.3.1. Heartbleed ... 5
2.3.2. Syn Flooding ... 5
2.3.3. TCP Reset ... 5
2.3.4. FREAK ... 5
3. Cryptography ... 6
3.1. Protocols ... 6
3.2. Standards ... 6
3.3. Symmetric-Key Cryptography ... 6
3.3.1. Block Cipher ... 7
3.3.2. Stream Cipher ... 8
3.3.3. Overall Construction ... 8
3.4. Asymmetric Cryptography ... 8
3.4.1. Signature Schemes ... 8
3.4.2. Public Key Infrastructure (PKI) ... 9
3.4.3. Signatures vs MACs ... 9
3.4.4. Establishing a shared key ... 9
3.4.5. Asymmetric Encryption ... 10
3.4.6. Overall Construction ... 10
3.5. Maths <3 ... 10
3.6. Quantum Cryptography ... 11
3.6.1. Quantum Key Distribution ... 11
3.6.2. BB84 (By example of polarized light) ... 11
4. IPsec ... 12
4.1. Protocols ... 12
4.1.1. Authentication Header (AH) ... 12
4.1.2. Encapsulating Security Payload (ESP) ... 13
4.2. Modes of Operation ... 14
4.2.1. Transport vs. Tunnel Mode ... 14
4.2.2. Transport Mode ... 14
4.2.3. ESP Tunnel Mode ... 14
4.3. Security Associations ... 14

4.3.1. SAD and SPD . . . 15
4.3.2. Handling of outbound packets . . . 15
4.3.3. Handling of inbound packets . . . 15
4.4. Internet Key Exchange v2 (IKEv2) . . . 15
4.4.1. Initial Exchanges . . . 16
4.4.2. AUTH . . . 16
4.4.3. Child SA Exchange . . . 17
4.4.4. INFORMATIONAL . . . 17
4.4.5. Key Derivation (Function) . . . 17
4.4.6. Algorithms . . . 18
4.5. IPsec & NAT . . . 18
4.5.1. Problems . . . 18
4.5.2. NAT-T and IPsec . . . 18
5. WireGuard . . . 19
5.1. Crypto stack . . . 19
5.2. Setup . . . 19
5.3. Protocol Phases . . . 19
5.3.1. Connection establishment . . . 19
5.3.2. First Handshake Message . . . 19
5.3.3. Second Handshake Message . . . 19
5.3.4. Key Derivation . . . 20
5.4. IKEv2/IPsec vs WireGuard . . . 20
6. DNSSEC . . . 20
6.1. Dan Kaminsky DNS Vulnerability (cache poisoning) . . . 20
6.2. DNS Security Extensions (DNSSEC) . . . 21
6.2.1. RRsets . . . 21
6.2.2. Zone Signing Keys (ZSK) . . . 21
6.2.3. Validation . . . 21
6.2.4. Key Signing Keys (KSK) . . . 21
6.2.5. Overview . . . 22
6.2.6. Resource Records . . . 22
6.2.7. Denial of existence . . . 23
6.2.8. DNS-Based Authentication of Named Entities (DANE) . . . 23
6.2.9. Deployment . . . 24
6.2.10. Root zone signing key . . . 25
7. Network Access Control (NAC) . . . 25
7.1. Port-Based Network Access Control - IEEE 802.1X . . . 25
7.1.1. Authentication . . . 26
7.2. Extensible Authentication Protocol (EAP) . . . 26
7.2.1. Native mode . . . 26
7.2.2. Tunneled mode . . . 27
7.3. Secure Device Identity (DevID) - IEEE 802.1AR-2018 . . . 28
7.3.1. Use of DevIDs . . . 28
7.4. Media Acccess Layer Security (MACsec) - IEEE 802.1AE . . . 29
7.4.1. SC and SA Identifiers . . . 29
7.4.2. Frame Format . . . 30
7.4.3. MACsec Key Agreement (MKA) - IEEE 802.1X . . . 31
7.5. Trusted Network Connect . . . 32

1. INTRODUCTION

OSI Stack:

Communication layers	*Security protocols*
Application layer	Web Application Security, VoIP Security, SW Security
Transport layer	TLS
Network layer	IPsec
Data Link layer	L2TP, IEEE 802.1X, IEEE 802.1AE, IEEE 802.11i (WPA2)
Physical layer	Quantum Key Exchange

The CIA triad is a foundational information-security model stating that systems should protect:

Confidentiality: Keeping information secret

Integrity: Keeping information correct and unaltered

Availability: Ensuring information and systems remain accessible

Additional desired properties include:

Authenticity: Adversary cannot claim to be someone else

Accountability/Non-repudiation: Actions can be traced to actor

2. TLS

SSL and TLS are protocols for internet handshakes and encrypted transmission. Secure Socket Layer (SSL) came first, then after v3.0 it became Transport Layer Security (TLS), currently v1.3. People still use SSL as a term, even though technically it's now TLS.

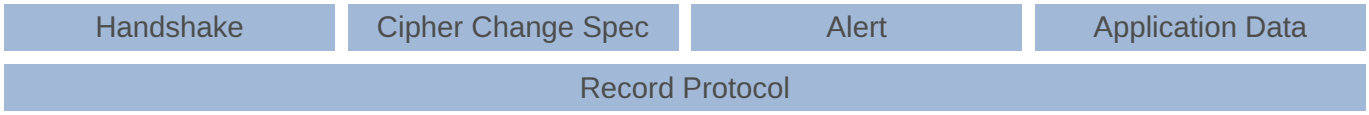
TLS 1.2: Introduced authenticated encryption, which is the most modern form of encryption. It's a more flexible protocol, lots of TLS extensions, the hashing function has been improved.

TLS 1.3: The protocol has been redesigned to make it faster. A lot of the old ciphers have been removed to make it more secure. It only supports authenticated encryption (AEAD).

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384

Encryption protocol	Key exchange algorithm	Signature algorithm	Bulk encryption algorithm	Message Authentication Code (MAC)

2.1. SPECIFICATION



2.1.1. Record protocol

All TLS packets are sent using the record protocol. This header is attached to every single message.

Header			Data
Type (1B)	Version (2B)	Length (2B)	…

Type: which kind of TLS message is sent (20,21,22,23)

Version: which TLS version is used

Length: how long is the payload of the message

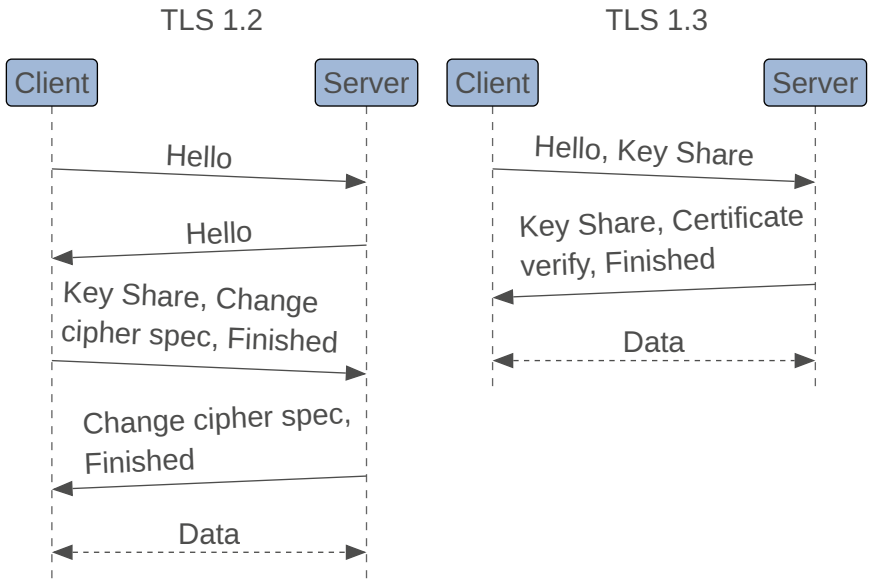
2.1.2. Subprotocols

Protocol	*Definition*
20 ChangeCipherSpec	The sender has sufficient information and is switching to encryption or to a new cipher suite (sent very rarely). By starting encryption for the first time, it is saying that the handshake has been successful, and encryption can start
21 Alert	An SSL-level alert notification (example close_notify). Not necessarily an error. Example: problem with the handshake, received data length is wrong, warning.
22 Handshake	Messages sent right at the beginning (example ServerKeyExchange). Eg. To establish a cipher, to establish our keys
23 Application Data	Opaque application data. Any applications data once we start the encryption is going to be sent under protocol 23

2.1.3. Handshake

The TLS handshake protocol is used to establish parameters for the remainder of the session. It must:

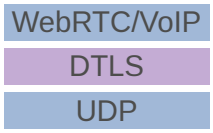
– Agree ciphers and protocols
– Establish shared secrets
– Authenticate server and client
– Be robust to tampering and attacks

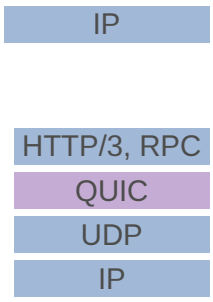


2.2. MODERN TLS VARIANTS

2.2.1. DTLS

– TLS on top of UDP
– Used for WebRTC, VoIP
– Does not provide ordering of packets
– Packets may drop, retransmission is delegated to application





2.2.2. QUIC

– TLS 1.3 + UDP
– Provides reliable communication
  – Stateful
  – Multiple streams
– Used for HTTP/3, RPC

2.3. ATTACKS

2.3.1. Heartbleed

The Heartbeat extension is a keep-alive feature of TLS. The `heartbeat_request` message includes payload length, payload and padding fields. The Heartbleed bug is an implementation flaw in that extension.

1) Send message that indicates the maximum payload length (64 KB) that only includes the minimum payload (16 bytes)
2) Receive almost 64 KB of random memory that the server returns
3) Profit (look for private keys, auth cookies, etc.)

Countermeasures: Update OpenSSL

2.3.2. Syn Flooding

Idea: Fill the TCB (Transmission Control Block) queue storing the half-open (never ACKed) TCP connections so that there will be no space to store TCB for any new half-open connections. Basically the server cannot accept any new SYN packets.

1) Use random source IP addresses; otherwise the attacks may be blocked by the firewalls.
2) Spam TCP requests
3) Profit (sleep well knowing you're a menace to society)

The SYN+ACK packets sent by the server may be dropped because forged IP address may not be assigned to any machine. If it does reach an existing machine, a RST packet will be sent out, and the TCB will be dequeued.

Countermeasures: SYN Cookies
– After a server receives a SYN packet, it calculates a keyed hash (H) from the information in the packet using a secret key that is only known to the server.
– This hash (H) is sent to the client as the initial sequence number from the server. H is called SYN cookie.
– The server will not store the half-open connection in its queue.
– If the client is an attacker, H will not reach the attacker.
– If the client is not an attacker, it sends H+1 in the acknowledgement field.
– The server checks if the number in the acknowledgement field is valid or not by recalculating the cookie.

2.3.3. TCP Reset

Spoof RST Packet to break up a TCP connection between Alice and Bob.

The following fields need to be set correctly: Source IP address, Source Port, Destination IP address, Destination Port, Sequence number (within the receiver's window)

2.3.4. FREAK

– Man-in-the-middle (MitM) downgrades used TLS cipher suite to use export RSA
– MitM then factors 512 bit RSA to get session key

TODO:
https://ciphersuite.info/cs/TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256/

3. CRYPTOGRAPHY

Parties: The abstract entities in cryptographic protocols (servers, switches, humans)

Attacker model: Defines the capabilities of the attacker (computational limits, corruption)

Secure channel: Provides Confidentiality (we allow length leakage) and Authenticity (we don't care about liveness)

Authentic channel: Only provides Authenticity

3.1. PROTOCOLS

TODO:
W2 slides 18

3.2. STANDARDS

Don't roll your own crypto, except if you work at FAANG, they shall perish.

National Institute of Standards and Technology (NIST) publishes
– Special Publications (SP), guidelines and recommendations
– Federal Information Processing Standards (FIPS), standards for US government

Internet Society (ISOC)
– Organizes working groups for TCP/IP
– Publishes RFCs

3.3. SYMMETRIC-KEY CRYPTOGRAPHY

Insecure channel: Vulnerable to MITM Attacks

Shared key: Secure means the adversary has no information about the shared key. Establishing a shared key is non-trivial
– Pre-shared key (Code book)
– Quantum key exchange
– Public Key Cryptography

Message Authentication Codes (MAC): Provides authentic channel over an insecure channel (a short piece of information used for authenticating and integrity-checking a message, like a signature but symmetric). Vulnerable to length-extension-attacks.

Cryptographic hash function: Should be collision-resistant and one-way. Good: SHA2, SHA3, Bad: MD5, SHA-1

Symmetric Encryption: Provides secure channel over an authentic channel

Hash-based MAC (HMAC): Fixes length-extension-attack vulnerability.

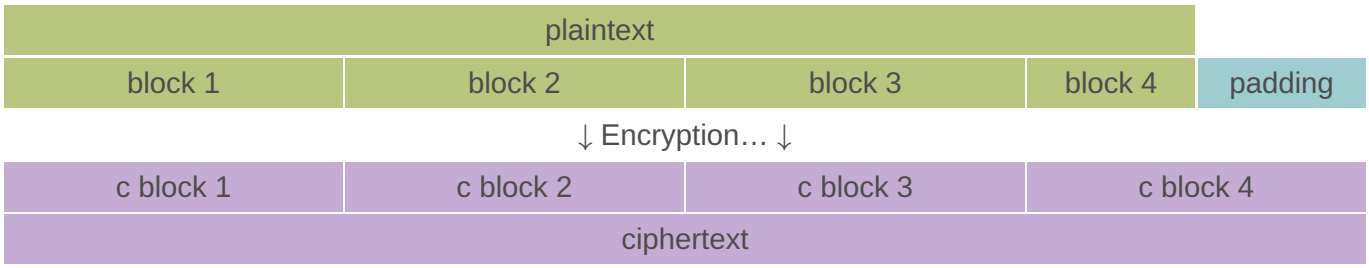
$$\mathrm{Hash}(\mathrm{key} \oplus \mathrm{opad} \mid \mathrm{Hash}\ (\mathrm{key} \oplus \mathrm{ipad} \mid m))$$

Block Size: Size of transformed/compressed data in the hash function for each iteration.

Message Digest Size: The output size of the hash function, also called hash value.

3.3.1. Block Cipher

Block ciphers take an input of a fixed size and return an output of the same size



– Block ciphers attempt to hide the transformation from message to ciphertext through confusion and diffusion
– Most block ciphers are SP-Networks

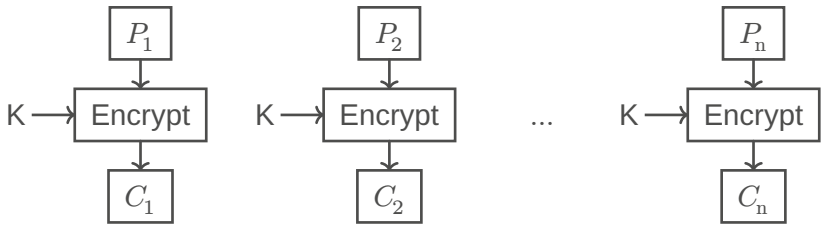
The Advanced Encryption Standard (AES) is an SP-Network

– Almost everything uses AES
– There are others (e.g. Feistel Ciphers)

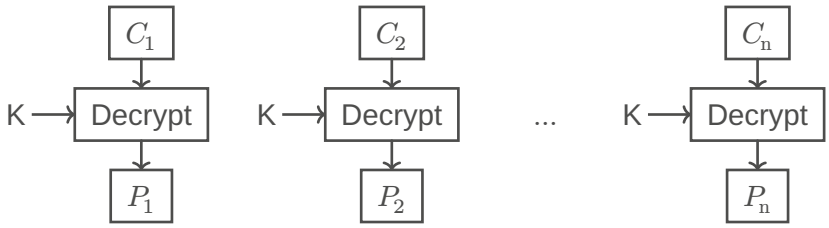
3.3.1.1. Electronic Code Block (ECB)

– Just encrypt each block one after another with same key
– Weak to redundant data divulging patterns
– Electronic codebook is not recommended!

Encryption



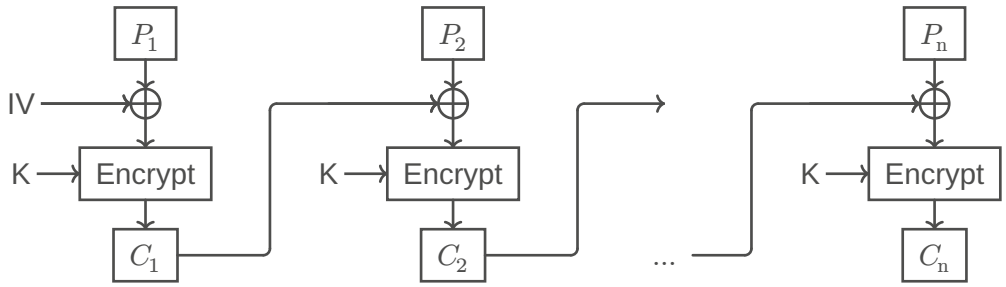
Decryption



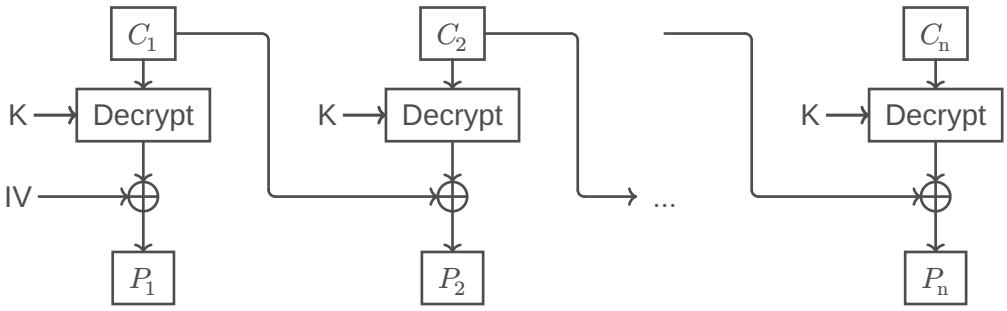
3.3.1.2. Cipher Block Chaining (CBC)

– XOR the IV with the first input, then XOR the output of each cipher block with the next input
  – Not parallelizable
  – It is better than ECB but not perfect

Encryption



Decryption



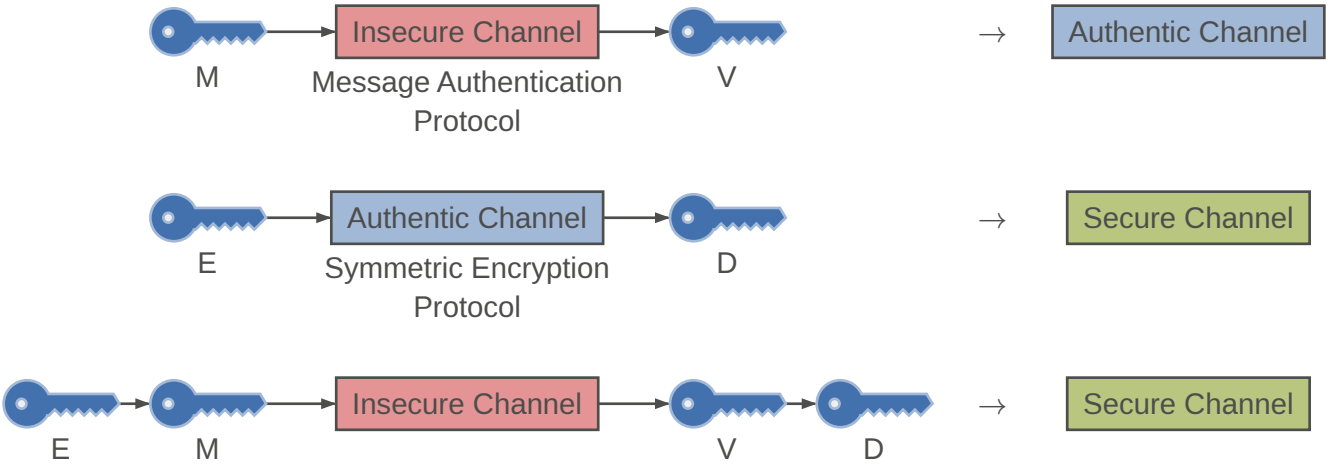
3.3.2. Stream Cipher

Inputs: Secret key + Public nonce (aka IV)

We can approximate a one-time pad by generating an infinite pseudo-random keystream

– Stream ciphers work on messages of any length (no blockification required)
– The nonce guarantees that each keystream is unique, even if the same key is reused

3.3.3. Overall Construction



Authenticity: E.g. HMAC-SHA256

Confidentiality: E.g. AES-CBC

Authenticity & Confidentiality: E.g. HMAC-SHA256-AES-CBC, AES-GCM

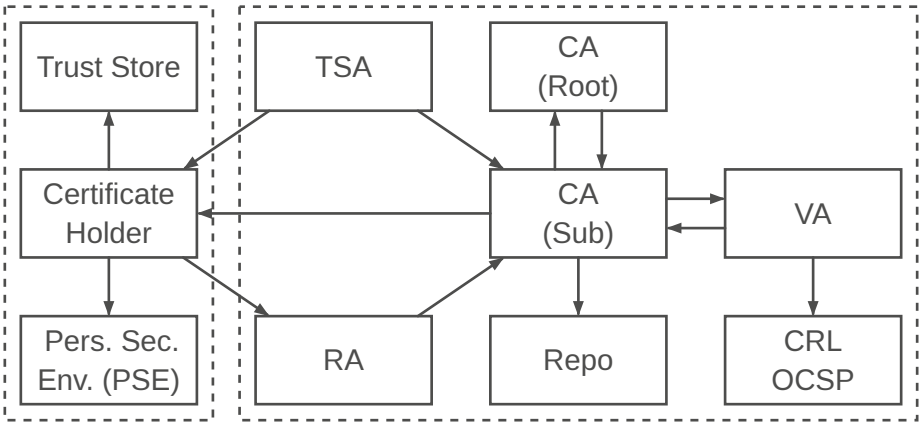
3.4. ASYMMETRIC CRYPTOGRAPHY

3.4.1. Signature Schemes

Private key used to **sign** messages. **Public key** used to **verify** signature. Signatures provide authenticity and non-repudiation. Examples: RSA, DSA, ECDSA

Signing is (often) computationally heavy. Some signature schemes natively only support small messages (naive RSA). Solution: Sign the hash of the message instead of the message itself

3.4.2. Public Key Infrastructure (PKI)



Each party has a private key and every other party knows their public key, e.g. via a registry. X.509 certificates and Certificate Authorities (CA) are used.

PKI provides an authentic channel over an insecure channel.

3.4.3. Signatures vs MACs

Signatures	***MACs***
Authenticity, non-repudiation	Authenticity only (key shared)
Slow (asymmetric)	Fast (symmetric)
Often: Use with long term keys	Often: Used with ephemeral keys

3.4.4. Establishing a shared key

Has to be established over an authentic channel.

Good pratice: Ephemeral keys, randomly generated for each session. Provides Independence between sessions and limits impact if compromised.

Problem: Authenticity, you need to know with whom we agree on a key, otherwise MitM possible. One cannot use MACs (no shared key available).

Key Exchange Protocol (KEX): Both parties contribute to the key, often the same protocol for both parties (e.g. DH KEX)

Key Encapsulation Mechanism (KEM): One party generates the key, Key is wrapped (= encrypted) and sent to the other party

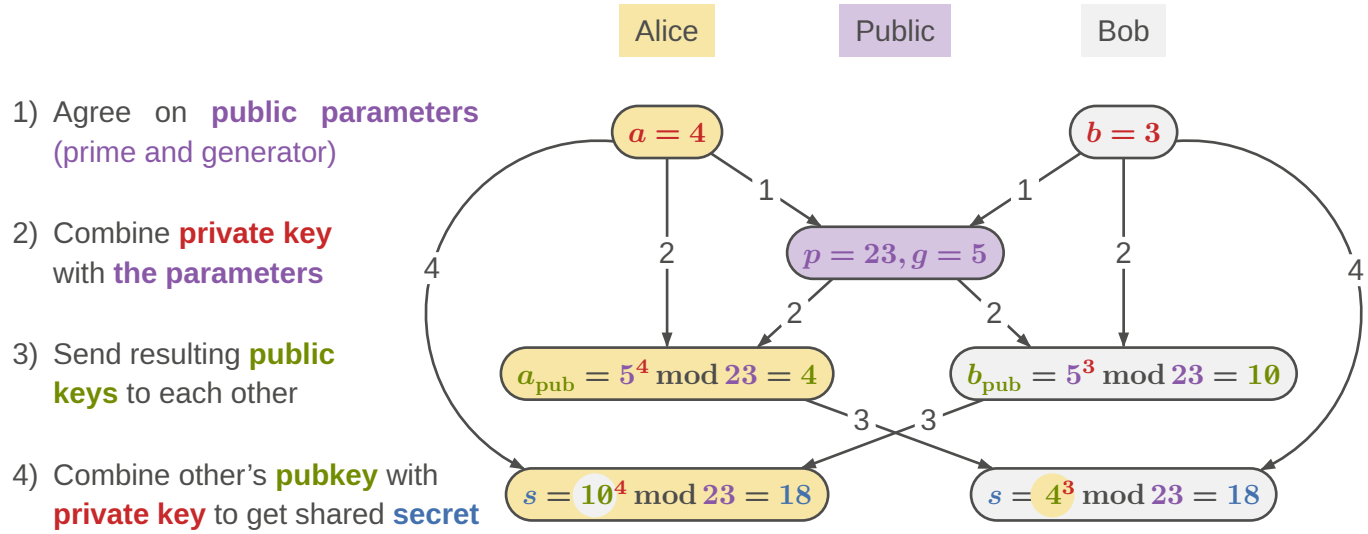
> ***TODO:***
> W2 diagram slides 46

3.4.4.1. Diffie-Hellmann

– With Diffie-Hellman, two parties can jointly agree a shared secret over an insecure channel
– Every communication handshake on the internet is powered by DH
– We are exchanging some parts of the mathematical key and then we secretly create the key ourselves

Process:
– p is usually 4096 or 6144 bits
– g is a primitive root of p
– **private keys** are values between 1 and p
– The **shared secret** (often called the pre-master secret) serves as the foundation for deriving all subsequent session keys.

  - The raw shared secret is not used directly for encryption, as it is typically a very large integer (e.g., 4096 bits in RSA or DH) and may not have uniform entropy.
  - We derive a master secret using a hashed-key derivation function (HKDF), for example the SHA-256 hash function
- The only way to find **a** or **b** is to solve the Discrete Logarithm Problem.



Elliptic-Curve Diffie Hellman (ECDH) is becoming the standard nowadays due to shorter keys.

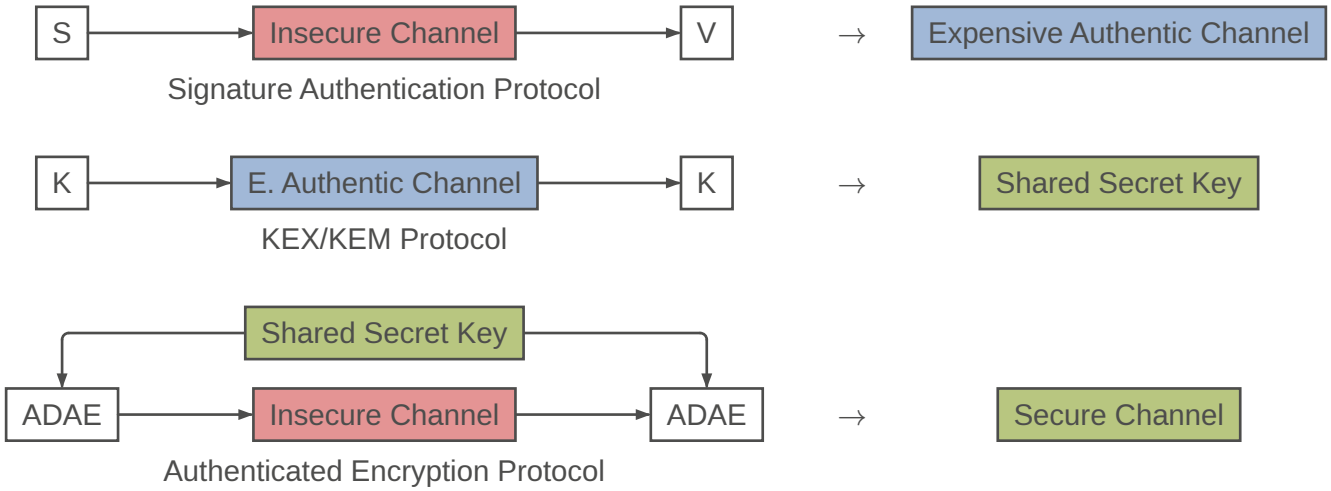
3.4.5. Asymmetric Encryption

- Public key used to encrypt messages
- Private key used to decrypt messages

E.g. RSA (factorization) or ElGamal (discrete logarithm)

TODO:
W2 slides 48,51

3.4.6. Overall Construction



3.5. MATHS <3

TODO:
W2, merge with DigCod

3.6. QUANTUM CRYPTOGRAPHY

Quantum Cryptography: Using quantum effects to do cryptography, All parties have access to quantum systems. Example: Quantum Key Exchange BB84

Post-Quantum Cryptography: Cryptography secure even if quantum computers exist, the adversary has access to a quantum computer, honest parties are (often) classic. Example: ML-KEM

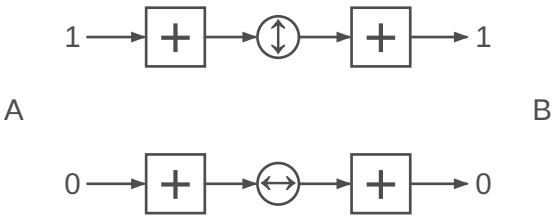
A big enough quantum computer could
– break known **asymmetric** schemes (Shor algorithm) e.g. RSA (factoring) and ElGamal, DH, ECDSA (discret log.)
– not break **symmetric** schemes e.g. Hash functions (SHA-2) and AES-256

3.6.1. Quantum Key Distribution

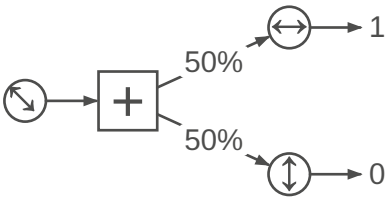
QKD's security is based on physical properties of quantum systems instead of hard math problems.

3.6.2. BB84 (By example of polarized light)

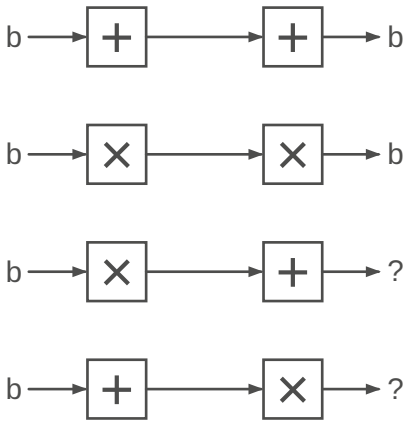
Signals sent



Randomization of output when using wrong polarization



Possible outcomes



No cloning theorem: One cannot duplicate photons. This prevents the idea of duplication and then measuring one while sending the other along.

Simplified protocol: For $1..k$:

1) Alice selects random bit b
2) Alice sends b using random base (+ or x)
3) Bob measures photon with random base (+ or x)

Bob and Alice exchange information on used bases over authentic channel
- They know which photons Bob measured correctly
- They have a common bit-string
- Errors imply listeners

3.6.2.1. Eavesdropping

Without Eavesdropping (Theoretical error rate: 0%):

50% chance Alice and Bob disagree on base	Bob gets correct bit

With Eavesdropping (Theoretical error rate: 25%):

50% chance Alice and Bob disagree on base	50% chance attacker guesses right base	
	Wrong base but Bob gets correct bit	Wrong base and Bob gets wrong bit

4. IPSEC

A collection of protocols that allow secure communication over IP networks. Provides confidentiality and/or authenticity and integrity of (parts of) IP packets.

4.1. PROTOCOLS

A collection of protocols that allow secure communication over IP networks:

Security Architecture for the Internet Protocol: RFC 4301 Defines basic architecture and requirements for IPsec gateways. In particular: Security Associations Management, e.g. “MUST support ESP”, z.B. “MAY support AH”

Authentication Header (AH): RFC 4302 Protocol to protect the integrity/authenticity of both payload and header data

Encapsulating Security Payload (ESP): RFC 4303 Protocol to protect the confidentiality and integrity/authenticity of payload data

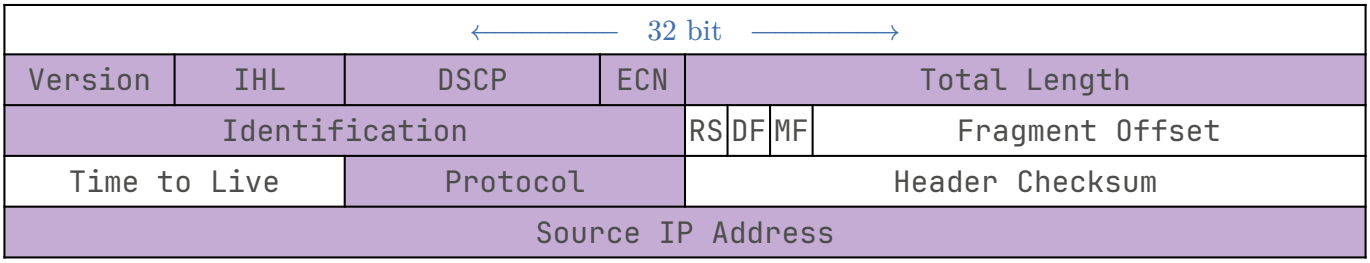
Internet Key Exchange v2 (IKEv2): RFC 7296 Protocol for manual and automated key exchange
Relevant for creating Security Associations used in AH and ESP

4.1.1. Authentication Header (AH)

Protects the integrity/authenticity of the payload and parts of the IP header (purple). Provides (optional) replay protection, does not provide confidentiality.

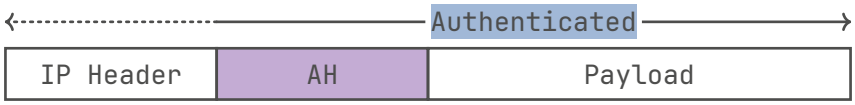
Protocol Field in the IP header: 0x33

Protected IP Header:



⟵ 32 bit ⟶
Destination IP Address
Options (if IHL > 5)

IP Packet with AH:



Authentication Header:

⟵ 32 bit ⟶		
Next Header	Payload Len	RESERVED
Security Parameters Index (SPI)		
Sequence Number Field		
Integrity Check Value - ICV (variable)		

Field	*Definition*
Next Header (8 bit)	ID of the protocol of the payload (eg. TCP/UDP)
RESERVED (16 bit)	All zeroes
Security Parameters Index (SPI) (32 bit)	Identifies the Security Association (SA), which in turn defines security attributes like Keys and Authentication algorithms
Sequence Number Field (32 bit)	For replay protection
Integrity Check Value - ICV (variable) (32 bit)	e.g. HMAC_SHA2_512_256

The sender increments the sequence number by one for every packet sent., the receiver stores the highest sequence number received so far.
– Too old or already received packets are rejected
– “Too old” is defined via the anti-replay window (default: 64)

The sequence number (32-bit value) is tracked per security association (SA)

– ***TODO:***
max 232-1 packets can be sent per SA

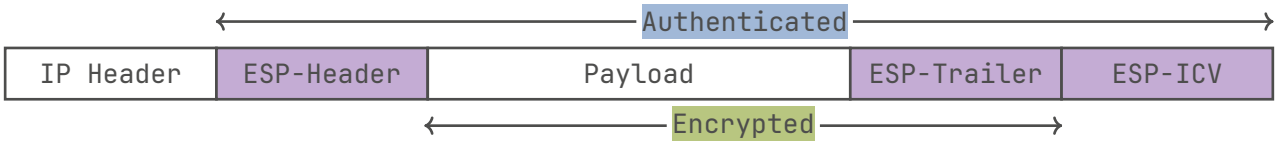
– Periodically a new SA is built

Extended Sequence Numbering are 64-bit values used for high-speed connections (>100 Gbps)

4.1.2. Encapsulating Security Payload (ESP)

Provides Confidentiality and authenticity of the payload. Authenticity (via ICV) and replay protection is optional (but recommended).

The IP header is not protected. Protocol Field in the IP header: 0x34



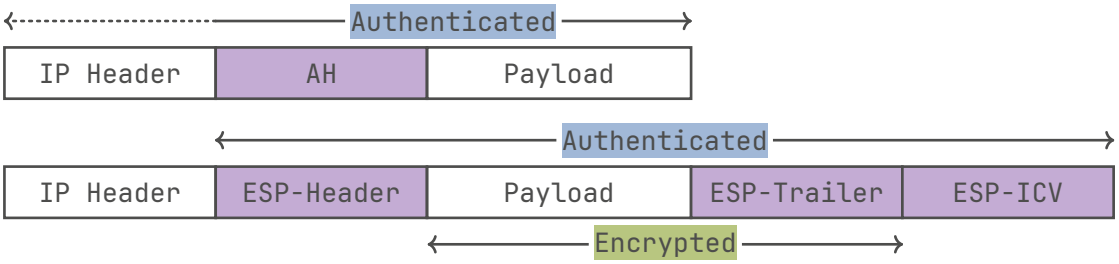
TODO:
W4 S14

4.2. MODES OF OPERATION

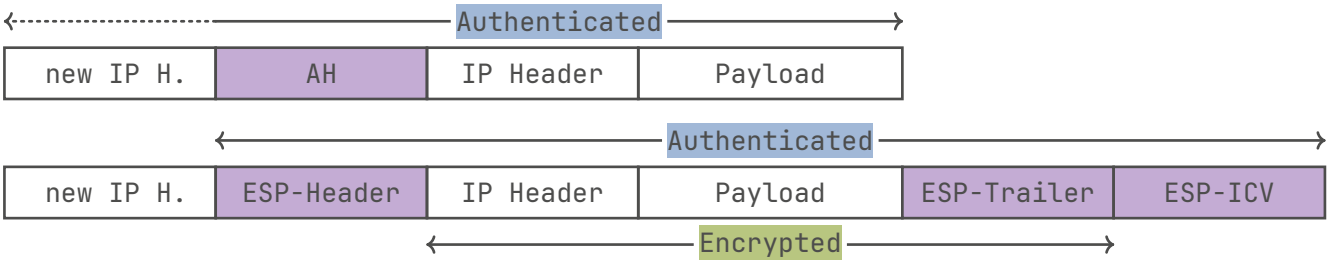
4.2.1. Transport vs. Tunnel Mode

Both AH and ESP have two modes of operation:

Transport Mode: Protects IP packets directly



Tunnel Mode: IP packets are packed into a new IP packet (as payload) which is then protected



4.2.2. Transport Mode

– The IP traffic is intended for the IPsec endpoints
– End-to-end security, but transparent for applications
– ESP: secure channel
– AH: authentic channel

4.2.3. ESP Tunnel Mode

Site-to-Site VPN:

– Tunnel between two IPsec gateways
– Transparent for endpoints. Independent of application
– Internal IP addresses remain confidential
– E.g. VPN tunnel between corporate networks

Client-to-Site VPN:

– Tunnel between an IPsec client and an IPsec gateway
– Internal IP addresses remain confidential
– E.g. Remote access to corporate network

4.3. SECURITY ASSOCIATIONS

For IPsec communication the peers need to share information on
– Cipher Suite, Key Length, Keys, …
– Protocol: AH or ESP
– Mode: Transport or Tunnel

– Sequence number counter
– Validity period of security associations

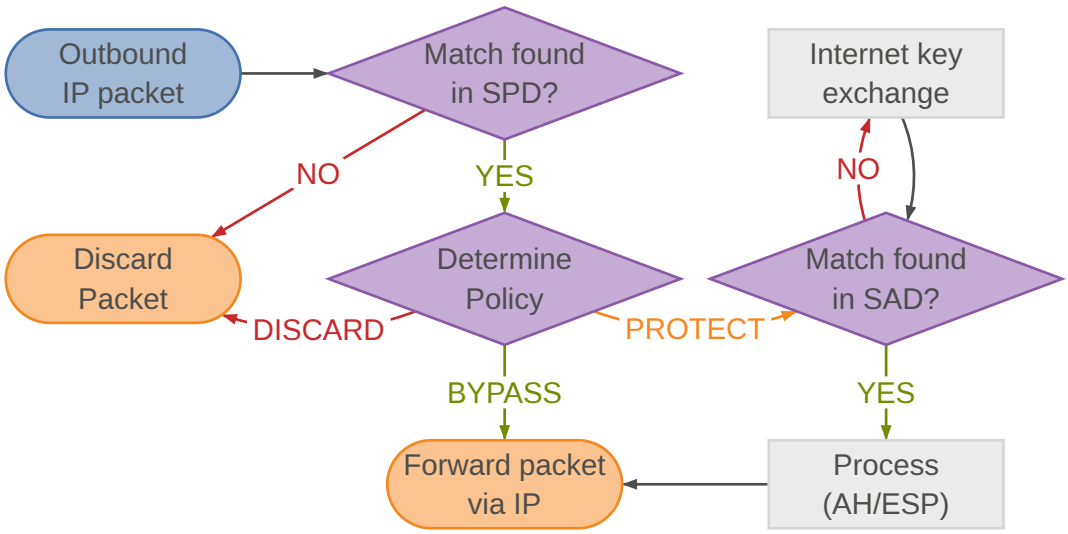
This information is stored in a security association (SA), Identified by a Security Parameters Index (SPI). Separate SAs for each communication direction, i.e. one per sender, and separate SAs for AH and ESP.

4.3.1. SAD and SPD

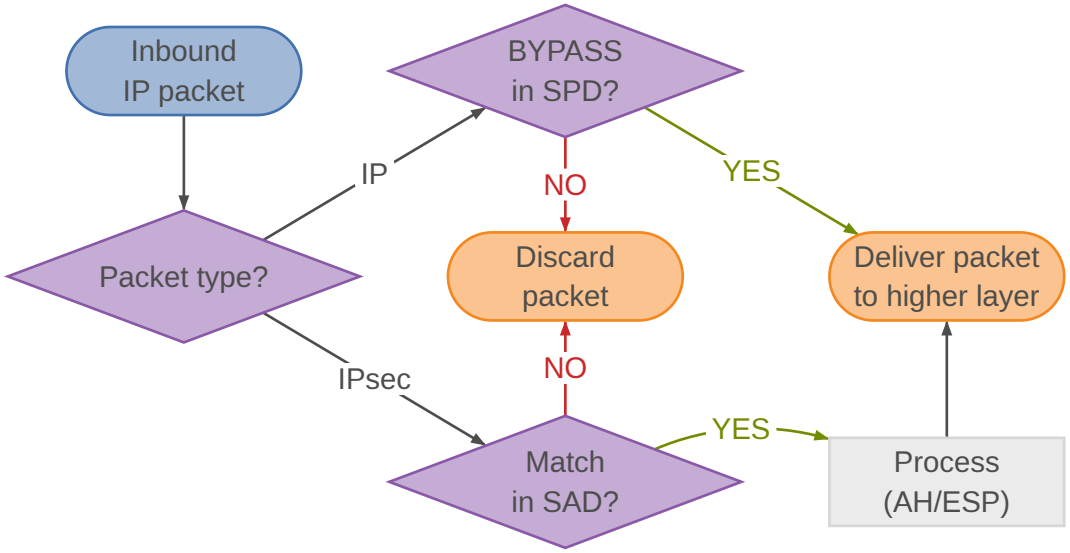
Security Policy Database (SPD): decides how to handle an IP packet
- Decision: Discard, Bypass, Protect
- Based on selectors: IP addresses, Next Layer Protocol, …

Security Association Database (SAD): holds all the security associations
- Incoming packets: SPI and possibly IP address point to SAD entry
- Outgoing packets: SPD entry points to SA

4.3.2. Handling of outbound packets



4.3.3. Handling of inbound packets



4.4. INTERNET KEY EXCHANGE V2 (IKEV2)

Protocol for mutual authentication and building of security associations between an Initiator and a Responder. RFC 7296

Goal: Initiator and Responder each have
– a security association for IKE transport (IKE-SA)
– security associations for ESP or AH traffic (CHILD-SA)

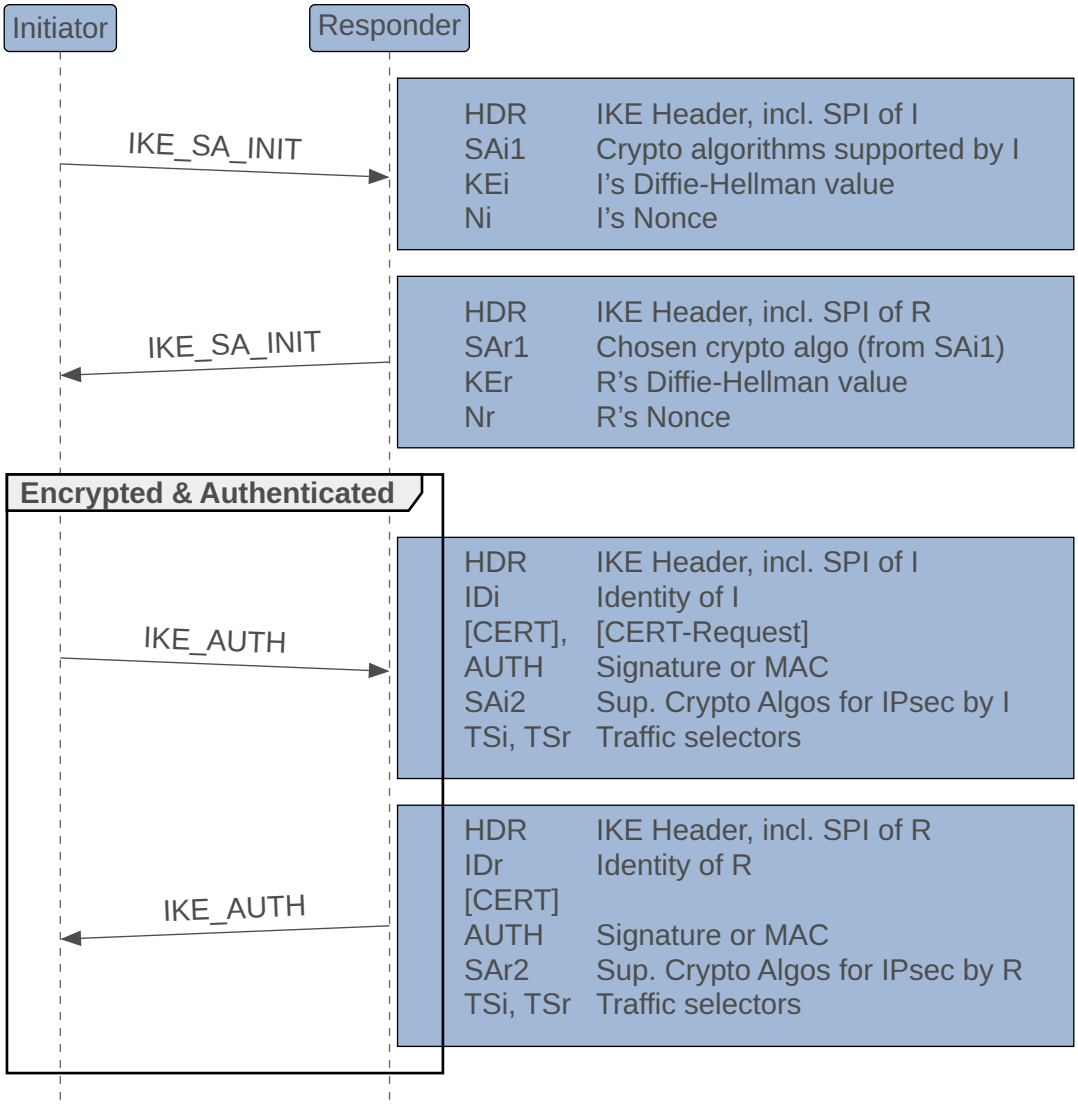
IKE messages are transmitted over UDP (port 500).

Protocol Phases:



4.4.1. Initial Exchanges

– The first two messages (IKE_SA_INIT) negotiate the security parameters and do a DH key exchange.
– The second pair of messages (IKE_AUTH) verifies the identities and establishes a first CHILD_SA.



4.4.2. AUTH

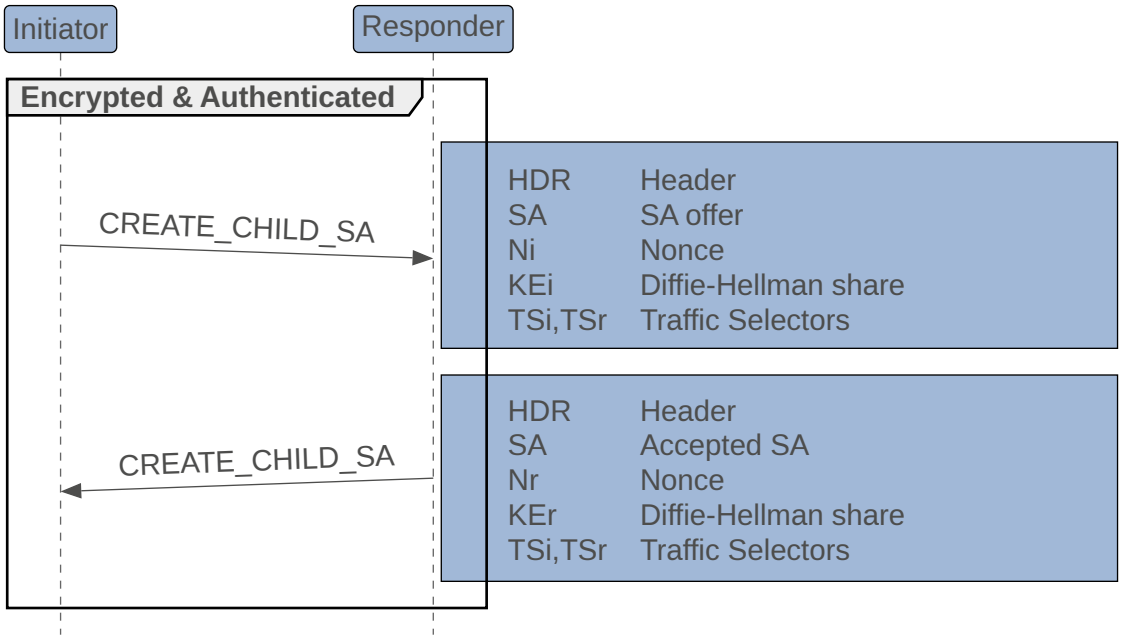
In the IKE_AUTH messages

– The identity of peers is verified by means of
  – Signatures (Certificates are sent in the messages)
  – Symmetric MAC if using a pre-shared-key
– The authenticity of the IKE_INIT messages is verified as well

– Instead of AUTH one can use Extensible Authentication (EAP)
  – Initiator omits the AUTH field to indicate this mode
  – Normally used to authenticate an initiator
  – Uses an additional exchange IKE_AUTH messages

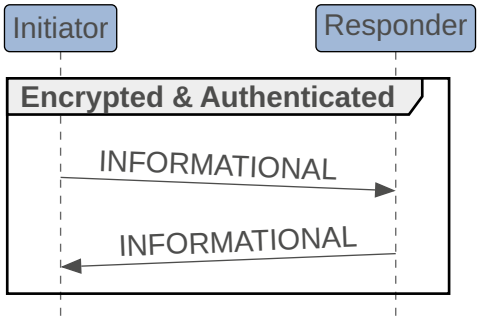
4.4.3. Child SA Exchange

Happens on new Child-SA or Rekeying of a Child-SA or Rekeying of the IKE-SA. Based on existing key material, optionally rekeying using DH.



4.4.4. INFORMATIONAL

For Control messages, e.g. error messages, event notification, deletion of SAs.

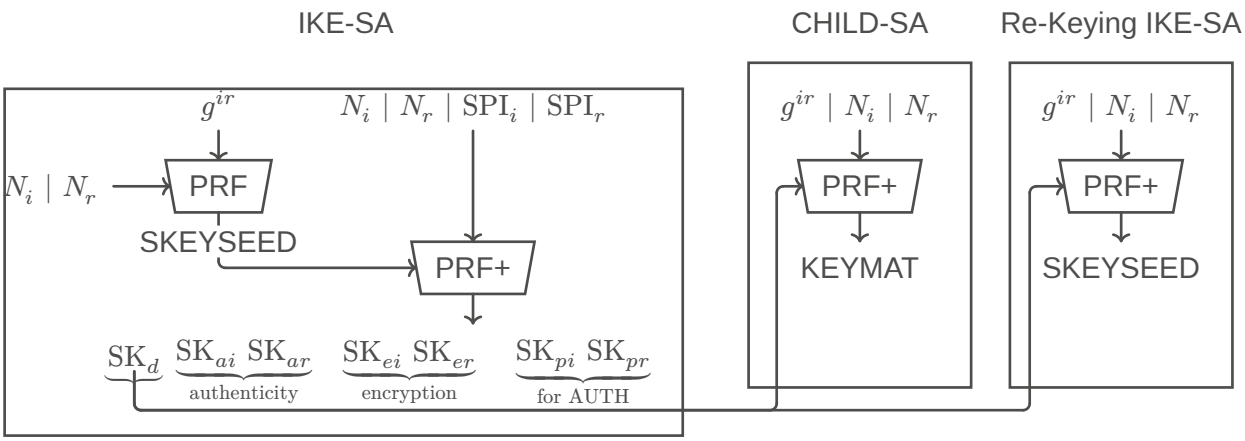


4.4.5. Key Derivation (Function)

PRF: Pseudo Random Function

KDF: Key Derivation Function

IKEv2 uses a PRF to derive key material, the PRF depends on cipher suite, e.g. AES-CMAC. Inputs to the PRF comes from Existing key material, the Nonce, and DH key exchange (ephemeral key).



TODO:
W4 S36

4.4.6. Algorithms

IPsec use various cryptographic algorithms (in IKEv2 negotiated (SAs)). RFC 8247 specifies “mandatory-to-implement” algorithms for IKEv2. RFC 8221 specifies algorithms for AH. Various RFCS define cipher suites with the goal of interoperability and a certain level of security, like RFC 4308 : Cryptographic Suites for IPsec (2005).

4.5. IPSEC & NAT

In one-to-one NAT, the NAT:
– replaces source address with its public address
– only works with one client

In Port Address Translation (PAT), the NAT:
– replaces source address with its public address
– Replaces (UDP/TCP) source port with unique free port
– Relays incoming packets to the right client based on the port number.

4.5.1. Problems

AH is incompatible as IP addresses are authenticated.

In ESP:
– PAT: The payload (UDP/TCP) is encrypted and cannot be changed by the NAT
– One-to-One NAT: TCP (and UDP in IPv6) has checksums that break when swapping out IP

In IKE:
– Using IP addresses as an identifier (IKEv1) causes issues.
– IKE responder needs to accept packets from a non 500 port.

4.5.2. NAT-T and IPsec

RFC 3948 : UDP-Encapsulation of IPsec ESP Packets
– UDP-Header between IP-Header and ESP-Header
– Source und destination port: 4500



RFC 4306 : IKEv2 can (implementation optional)
– Detect NAT (NAT_DETECTION_SOURCE_IP and NAT_DETECTION_DESTINATION_IP in INIT-msg)

– Negotiate UDP-Encapsulation for IKE and ESP
– Source und dst. port: 4500, reply to arbitrary port



5. WIREGUARD

Wireguard is a Fast, Modern, Secure, Free, Open Source, Best, Bestest VPN Tunnel.

5.1. CRYPTO STACK

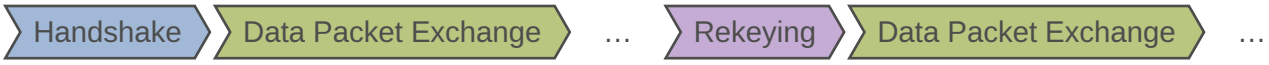
Symmetric:
– ChaCha20 for symmetric encryption, authenticated with Poly1305, using RFC 7539 ‘s AEAD construction
– BLAKE2s for hashing and keyed hashing, described in RFC 7693
– SipHash24 for hashtable keys
– HKDF for key derivation, as described in RFC 5869

Asymmetric:
– Curve25519 for ECDH, the public and private keys are for DH

5.2. SETUP

– Both parties generate a DH key pair.
– Exchange public keys with each other (done by an admin not the WireGuard protocol).
– Each party has a config file that maps the other party’s public key to their AllowedIPs
– When connecting a handshake of two messages is exchanged to verify the identity of the other party

5.3. PROTOCOL PHASES



5.3.1. Connection establishment

WireGuard uses the Noise_IK handshake pattern of the Noise Protocol Framework

I (Immediate): The initiator’s own static public key is transmitted (encrypted) inside Message 1, so the responder learns the initiator’s identity during the handshake itself.

K (Known): The initiator already knows the responder’s static public key before the handshake begins, because both sides configured each other’s public keys out-of-band as part of setup.

5.3.2. First Handshake Message

Initiator sends a message to the responder with:

Ephemeral (public key): Initiator’s one-time key for this session, in plaintext.

Encrypted Static (public key): Initiator’s long-term static public key, encrypted under a combination of the initiator’s ephemeral key and the responder’s pre-known static public key.

Encrypted Timestamp (TAI64N): 12 Byte, for replay protection

5.3.3. Second Handshake Message

Responder sends a message to the responder, after processing the first message, with:

Ephemeral (public key): Responder's one-time key for this session, in plaintext.

Encrypted Empty (payload): Proof of key possession → Encrypting an empty payload forces the responder to demonstrate it has correctly derived the same session key material as the initiator, without transmitting any additional identifying information.

5.3.4. Key Derivation

Conceptually a combination of DH and KDF as in IKE, but the overall design simpler than IKE.
– Long-term keys are used for authentication
– Ephemeral keys provide forward secrecy

Encryption: One key for each direction of communication

Post-quantum: Add pre-shared key into the mix

5.4. IKEV2/IPSEC VS WIREGUARD

Aspect	*IKEv2/IPSEC*	*WireGuard*
Connectivity	Connection-oriented	Connectionless
On Top Of	IP (alt. UPD)	UDP
Configuration Complexity	Medium	Low (static keys)
Configuration Possibilities	Wide	Limited
Speed	Fast	Generally faster
Quantum Resistance	Yes (Right Config)	Yes (with PSK enabled)
Which one to pick?	Frequent Mobile Roaming, Enterprise and Legacy Hardware, Regulatory Compliance	High-Performance Streaming and Gaming, Simple DIY and Self-Hosted VPNs, Resource-Constrained Hardware

6. DNSSEC

6.1. DAN KAMINSKY DNS VULNERABILITY (CACHE POISONING)

1) Bad guy sends a DNS query to the victim nameserver for the hostname it wishes to hijack
2) Knowing that the victim will shortly be asking ns1.bankofsteve.com (as directed from the root/GTLD servers) for an IP address, the bad guy starts flooding the victim with forged DNS reply packets. All purport to be from ns1.bankofsteve.com, but include the answer with the IP of badguy's fraudulent webserver
3) Root/GTLD servers provide referral to ns1.bankofsteve.com
4) Victim nameserver asks ns1.bankofsteve.com for the IP address of www.bankofsteve.com, and it uses query ID 1001 (one higher than the previous query)
5) The real nameserver provides a legitimate response to this query, with QID=1001. But if the bad guy has successfully matched the query ID in the step 2) flood, this legal reply arrives too late and is ignored
6) With the bogus IP address (of the bad guy's webserver) in cache it provides this poisoned answer to the requesting DNS client
7) Future DNS clients asking for www.bankofsteve.com will receive the same fraudulent answer

6.2. DNS SECURITY EXTENSIONS (DNSSEC)

RFC 4033 , RFC 4034 , RFC 4035

Provides **end-to-end protection through the use of digital signatures** that are created by responding zone administrators and verified by a recipient's resolver software.

Avoids the need to trust intermediate name servers and resolvers that cache or route the DNS records originating from the responding zone administrator before they reach the source of the query.

Consists of a set of **new resource record types and modifications to the existing DNS protocol**.

6.2.1. RRsets

Group all the records with the same **type** and the same **name** into a ***resource record set*** (RRset). The full RRset gets digitally signed and not the individual DNS records .



6.2.2. Zone Signing Keys (ZSK)

Each zone in DNSSEC has a zone-signing key pair (ZSK)
– The private key digitally signs each RRset in the zone
– The public key portion verifies the signature

A zone operator creates digital signatures (RRSIG) for each RRset using the private ZSK. The RRSIG is stored in the name server as a RRSIG record.

The zone operator also needs to make their public ZSK available by adding it to the name server in a DNSKEY record.

6.2.3. Validation

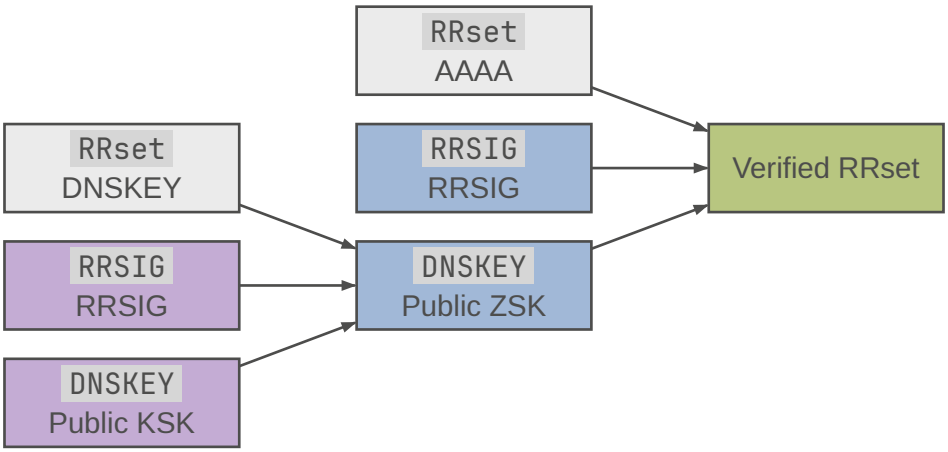
When a DNSSEC resolver requests a particular record type (e.g., AAAA), the name server also returns the corresponding RRSIG and DNSKEY records. The resolver can validate the response with the RRset, RRSIG, and public ZSK.

6.2.4. Key Signing Keys (KSK)

In addition to a zone-signing key, DNSSEC name servers also have a key-signing key (KSK). The KSK validates the DNSKEY records in exactly the same way as our ZSK secured our RRsets. The name server publishes the public KSK in another DNSKEY record.

TODO:
for rotating keys

6.2.5. Overview



6.2.6. Resource Records

6.2.6.1. DNS Public Key (DNSKEY)

The DNS Public Key used to sign the RRsets of a zone

```
switch.ch.      23248 IN  DNSKEY  256 3 13
                jVcj6ASFVqqUcjqtEjUG38+WVvbplnyelv9VBrSck5F5jzeionSDfn1F
                vreNQh37pLNpR2wlV+yrsLcpbM7qYQ==
switch.ch.      23248 IN  DNSKEY  257 3 13
                75SbTSctA8tgapqpGQSk8wpDl/ngvY4av6XJ/79ABB1cd1AxTNZDhl8y
                +8h0j8+TPmfwGD2CLvqnMQCsjyvmwQ==
```

Flags field
– 256 → Zone Signing Key (ZSK)
– 257 → Key Signing Key (KSK) with secure entry point (SEP) flag set

Algorithm field
– 5 → SHA-1 with RSA
– 8 → SHA-256 with RSA
– 10 → SHA-512 with RSA
– 13 → SHA-256 with ECDSA
– 14 → SHA-384 with ECDSA

6.2.6.2. Resource Record Signature (RRSIG)

Contains a public key signature over a resource record set.

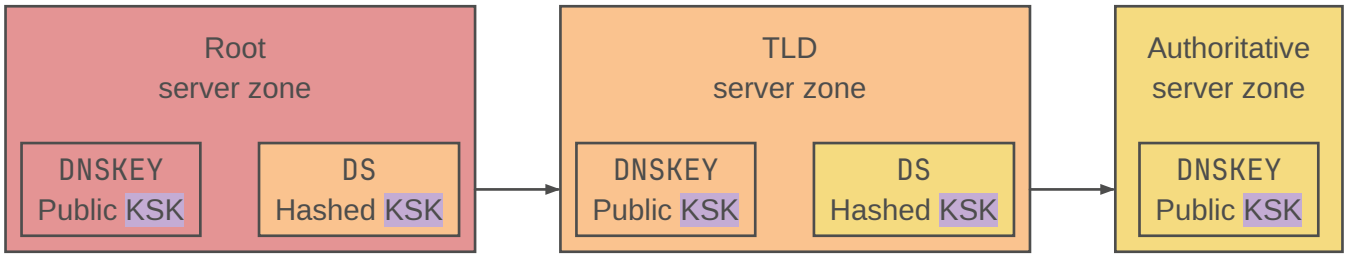
```
switch.ch.      22738 IN  RRSIG DNSKEY 13 2 86400
                20261107122148 20260922112148 26104 switch.ch.
                UoCt+fO0/Xs8j9SeKOqol1rTociNYnbhJDF1Up2bWLimYNEaV/x682Pt
                Iih9nNxTqXNjnRwY1U6Cv2s+T8NWkQ==
```

Signature Expiration and Inception Fields: The signature is not valid before Inception and after Expiration date.

Key Tag Field: Contains the key tag of the key which signed the RRset.

6.2.6.3. Delegation Signer Records (DS)

DNSSEC introduces a delegation signer (DS) record to allow the transfer of trust from a parent zone to a child zone. A zone operator hashes the DNSKEY record containing the public KSK and gives it to the parent zone to publish as a DS record.



Every time a resolver is referred to a child zone, the parent zone also provides a DS record. This DS record is how resolvers know that the child zone is DNSSEC-enabled.

To check the validity of the child zone's public KSK, the resolver hashes it and compares it to the DS record from the parent. If they match, the resolver can assume that the public KSK hasn't been tampered with, which means it can trust all of the records in the child zone.

This is how a chain of trust is established in DNSSEC.

6.2.7. Denial of existence

If you ask DNS for the IP address of a domain that doesn't exist, it returns an empty answer This is a problem if you want to authenticate the response, since there's no message to sign. DNSSEC fixes this by adding the NSEC and NSEC3 record types.

6.2.7.1. Next Owner Name (NSEC)

```
ns2.switch.ch. 179 IN NSEC  nsec3.switch.ch.
               A PTR AAAA RRSIG NSEC
ns2.switch.ch. 179 IN RRSIG NSEC 13 3 180
               20161112201311 20161021040550 25876 switch.ch.
               3mDDwKXFtgeJSUBe0WEvtVhwwjO2MPxa
               2KONRTGz6/24+v0qB0PMrKsaQVtK26Ls
               1Vmk/hbQCorRcIByLrglzQ==
```

Proof that there is no name between ns2.switch.ch. and nsec3.switch.ch.

Allows enumeration of complete zone data.

6.2.7.2. Next Owner Name in Hashed Order (NSEC3)

```
h9p7u7tr2u91d0v0ljs9l1gidnp90u3h.org. 691 IN NSEC3
               1 1 1 d399eaab
h9rsfb7fpf2l8hg35cmpc765tdk23rp6
               NS SOA RRSIG DNSKEY NSEC3PARAM ; flags: optout
h9p7u7tr2u91d0v0ljs9l1gidnp90u3h.org. 691 IN RRSIG NSEC3 7 2
               86400 20091202211702 20091118201702 5273 org.
               a+CC37hRM7yCFBaZn2SeRgY9h247GXptCuBYf45TwaoR
               xvBwTAXPT+UwZ/4hxwc2v7AR7ZZ8UOMiNJvYsl59eFW8
               Xtgws4/Aih0fJ2/O8yUHwI695fRf9PrpxXEpqzStjSZP
               5arJ1oldDAHcnxgLqdAMW6wnK1FNrslfJblJlmU=
```

Proof that there is no name between org. and ???.org.

Does not allow straight enumeration of zone data.

6.2.8. DNS-Based Authentication of Named Entities (DANE)

Through DANE the browser can check the TLSA record for a public fingerprint of a certificate that the user has marked as safe.

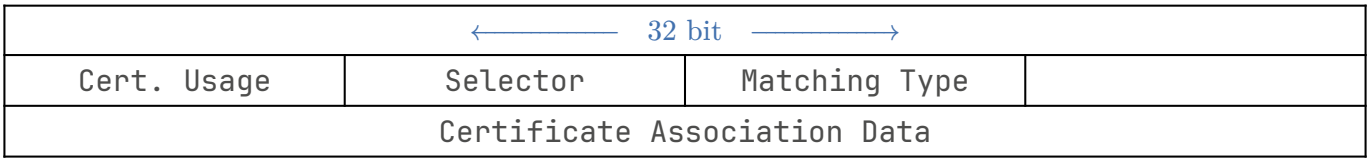
DANE, defined in RFC 6698 , is a protocol to allow X.509 certificates, commonly used for Transport Layer Security (TLS), to be bound to DNS names using DNSSEC, as a means to authenticate TLS client and server entities without a certificate authority (CA). The purpose of DANE is to replace reliance on the security of the CA system with reliance on the security provided by DNSSEC. It defines a new DNS record type, TLSA, that can be used for a secure method of authenticating SSL/TLS certificates.

TODO:
The TLSA RRs used by DANE store hashes of entire certificates or the public key (subject- PublicKeyInfo) or their complete encoding. There are four ways of establishing trust in the server certificate, which were given new acronyms with RFC 7218 :

– A CA certificate is referenced which must appear in the normally verified chain of trust (“CA constraint” or PKIX-TA).
– Reference the end entity certificate of the server whose chain of trust is otherwise normally verified (“service certificate constraint” or PKIX-EE).
– The trust anchor (CA) is referenced, which does not have to be known to the client beforehand - this also works for self-signed CAs (“trust anchor assertion” or DANE-TA).
– You reference the certificate of the server whose trust chain is not checked. This works with self-signed certificates (“domain-issued certificate” or DANE-EE). The names in the certificate do not even have to match the host name.

Since this in combination with the type of reference (certificate or PK) and the data to be transmitted (complete, SHA-256 or SHA-512 hash) results in 24 possible configurations, only certain combinations were recommended for use by RFC 7671 . This should simplify the implementation of clients and deployment as well.

A client locates TLSA RRs by the name `_<port>._<proto>.<domain>`, where `<proto>` is the transport protocol (tcp, udp, etc.) and `<port>` represents the port used (e.g. 443 for HTTPS).



Field	*Definition*
Cert. Usage (8 bit)	0 = CA Certificate Constraint 1 = Server Certificate Constraint 2 = Trust Anchor Assertion for Private CA 3 = Domain Issued Certificate (self-signed)
Selector (8 bit)	0 = Full Certificate 1 = Public Key Info (Public Key plus Key Type Information)
Matching Type (8 bit)	0 = Exact Match on Selected Content 1 = SHA-256 Hash of Selected Content 2 = SHA-512 Hash of Selected Content

TODO:
W5 S42..46

6.2.9. Deployment

DNSSEC’s adoption isn’t too wide-spread at the moment. :(

6.2.10. Root zone signing key

The root DNS zone has no parent, so how can we trust the integrity and authenticity of its information?

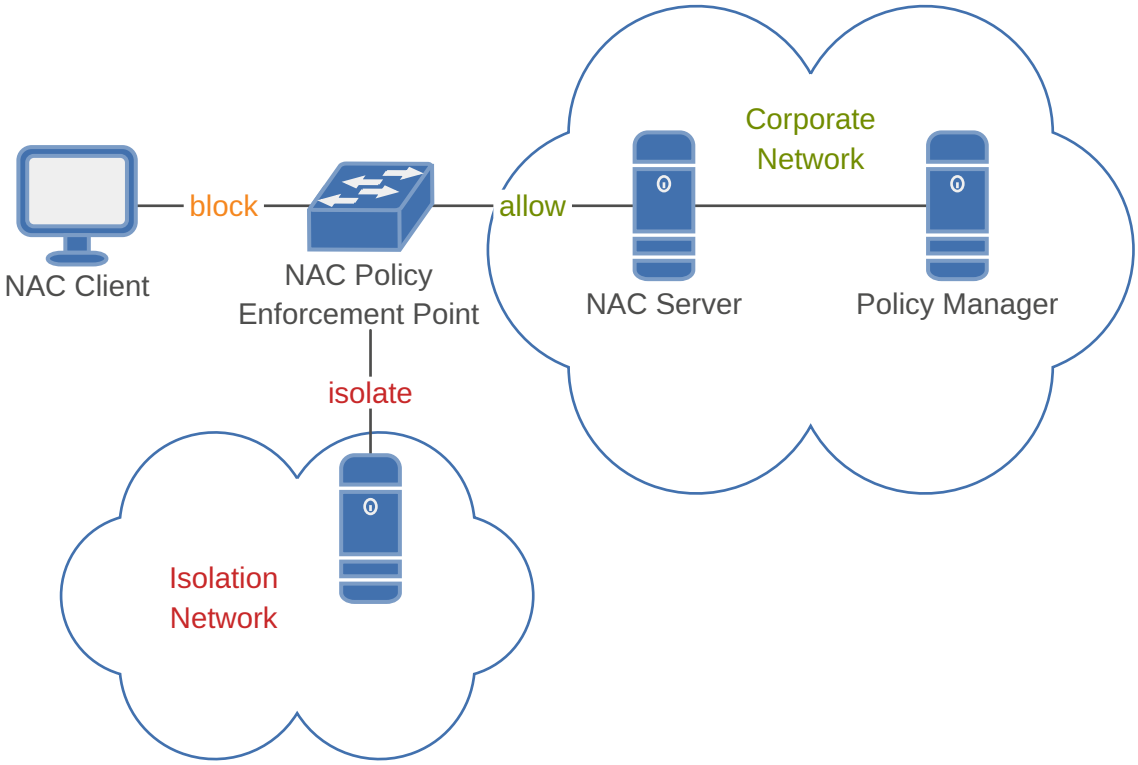
That's the purpose of the Root Signing Ceremony - a rigorous procedure around signing the root DNS zone's public keying information. The private signing key used in this process is quite literally the key to the entire DNSSEC-protected Internet. (I don't see any problem there…)

There are two geographically distinct locations that safeguard the root key-signing key: El Segundo, CA and Culpeper, VA. Both are secure facilities, and they contain redundant copies of the key.

7. NETWORK ACCESS CONTROL (NAC)

Network Access Control is an umbrella term for managing access to a network:
– Users logging into the network are authenticated and determined what data they can access and actions they can perform
– The health of the user's computer or mobile device is checked. E.g. It can then be put into the isolation network if the device misses updates and can fetch them there

It provides an authentic channel on the data link layer. Auth doesn't happen at authenticator (enforcement point) but at server.



There are many network access enforcement methods, the common ones include:
– Layer 2: IEEE 802.1X (switches and WLAN access points)
– Layer 2: Virtual local area networks (VLAN)
– Layer 3: VPN (IKEv2)
– Layer 4: VPN proprietary over TLS
– Firewalls
– DHCP management

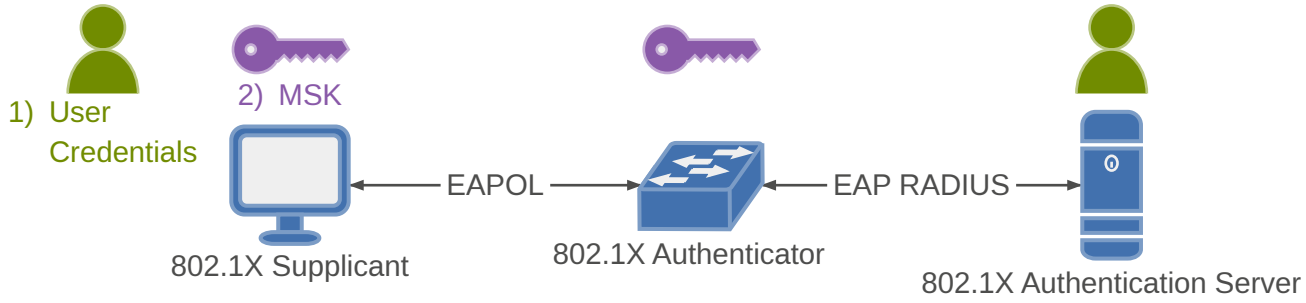
7.1. PORT-BASED NETWORK ACCESS CONTROL - IEEE 802.1X

EAP: Extensible Authentication Protocol

EAPOL: EAP over LAN

MSK: EAP Master Session Key

PAE: Port Access Entity. 802.1X Supplicants and Authenticators are both PAEs



MSK is established between supplicant and authenticator after successful authentication.

Authentication Server: The device that handles authentication, authorization, and accounting (AAA) services on a network. User accounts are created and stored; privileges and policies are applied.

Authenticator: The network device that is attempted to be accessed, e.g. wireless router/access point or a network switch. It has no idea of what EAP type is in use, or if the user's credentials are valid. It simply takes the unmodified EAP frame and encapsulates it within the RADIUS packet sent to the authentication server and authorizes the port if the authentication server tells it to.

Supplicant: The client device, such as a smartphone or laptop computer, that wants to access the network.

The actual EAP identity exchange and authentication is occurring between the supplicant and the authentication server.

7.1.1. Authentication

The authentication can be initiated by either authenticator or supplicant:

Authenticator: If the link state changes from down to up or periodically as long as the port remains up and unauthenticated. The switch sends an EAP-request/identity frame to the endpoint to request its identity

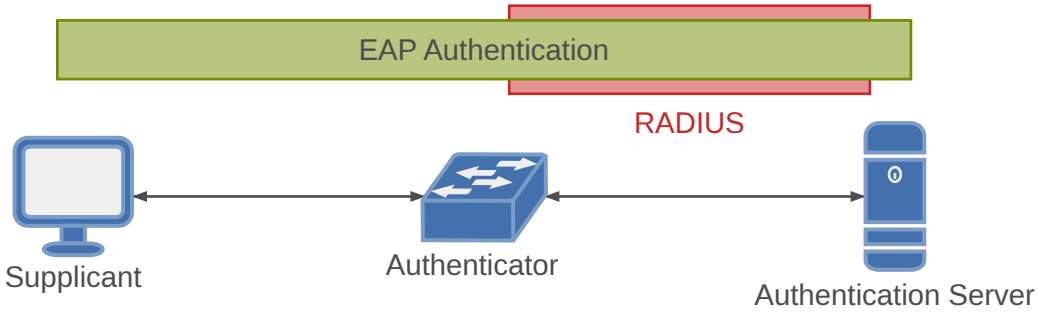
Supplicant: Triggers the authenticator to request an identity by sending an `EAPoL_Start` message at any time.

7.2. EXTENSIBLE AUTHENTICATION PROTOCOL (EAP)

RFC 3748 , RFC 5247

EAP Provides a protocol to encapsulate different authentication methods.

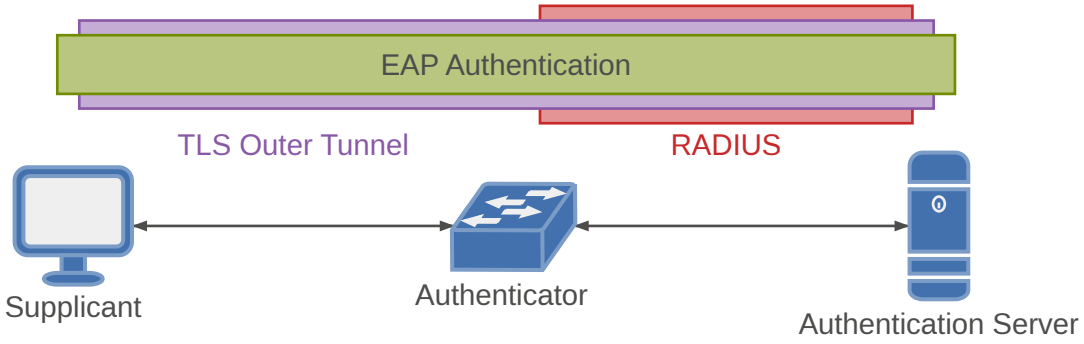
7.2.1. Native mode



EAP-MD5: Uses the MD5 message-digest algorithm to hide the credentials in a hash. It does not have a mechanism for mutual authentication. That means the server validates the client, but the client does not authenticate the server.

EAP-TLS: Uses TLS to provide the secure identity transaction through X.509 certificates, which provide strong authentication. It's an open IETF standard. Authentication occurs automatically, with no intervention by user, there's no dependency on user password. Autonomous enrollment benefits from this (IoT devices can each have their certificate instead of username/password)

7.2.2. Tunneled mode



Whereas the native EAP types send their credentials immediately, tunneled EAP types form an encrypted tunnel first and then transmit the credentials with native EAP within that tunnel.

With tunneled EAP types, there is a concept of inner and outer identities:

Inner identity: is the user or device's actual identity credentials sent with the native EAP protocol.

Outer identity: is typically set to "anonymous". It's the identity that is used between the supplicant and the authentication server for the initial TLS tunnel setup.

Cisco ISE is able to read that outer identity and use it to help make identity store selection decisions. Most supplicants (except e.g. Android) will hide this option from the end user, and only administrators see the outer identity.

7.2.2.1. Protected EAP (PEAP)

Forms an encrypted TLS tunnel between the client and server, using the X.509 certificate on the server. After the tunnel has been formed, PEAP uses one of the following EAP types as an inner method for authenticating the client using EAP within the outer tunnel.

MSCHAPv2: is the most common inner method, as it allows for simple transmission of username and password, or even computer name and computer password, to the RADIUS server. (Microslop Challenge Handshake Authentication Protocol Version 2)

PEAP: is capable of using EAP-TLS as an inner method but it is rarely used.

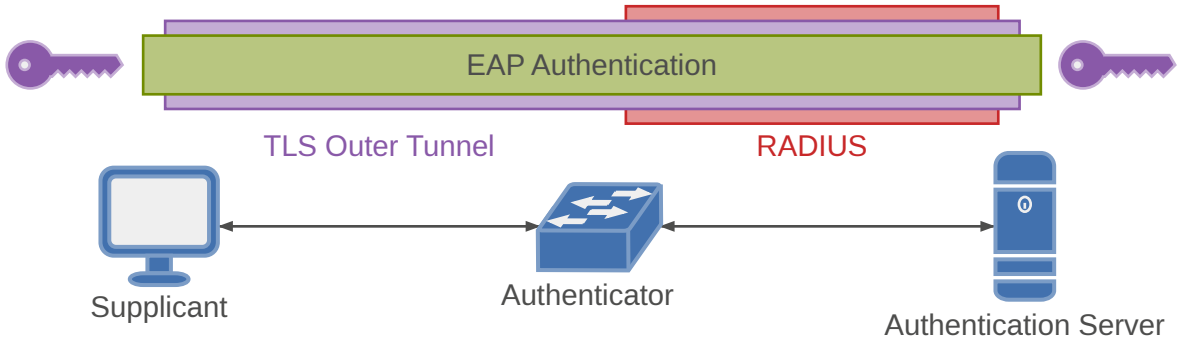
7.2.2.2. Flexible Authentication via Secure Tunneling (EAP-FAST)

Created by Cisco Systems as an alternative to PEAP to allow for faster re-authentications and support faster wireless roaming. FAST forms a TLS outer tunnel and then transmits the client credentials within this TLS tunnel. It has the ability to use Protected Access Credentials (PACs). A PAC can be thought of like a secure "cookie", stored locally on the host as "proof" of a successful authentication.

EAP-MSCHAPv2: is the most common inner method.

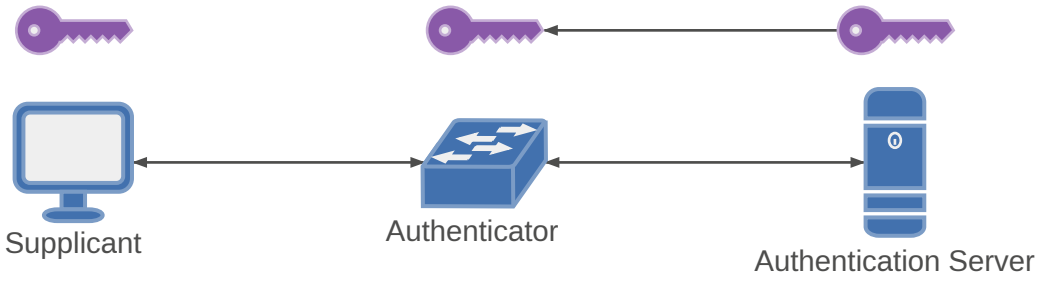
EAP-TLS: EAP-FAST is capable of using it as an inner method. This has become quite popular with EAP-Chaining (Authenticating Computer and User).

Step 1: Key established between supplicant and Authentication Server



But with TLS the authenticator can't see the key, so:

Step 2: Authentication Server sends key to Authenticator



7.3. SECURE DEVICE IDENTITY (DEVID) - IEEE 802.1AR-2018

DevID: Secure Device Identifier

IDevID: Initial Device Identifier
Created during manufacturing and cannot be modified Either reaches end of lifetime (certificate) or can be disabled

LDevID: Local Device Identifier
One or several may be created by network administrator

DevID Module: Hardware module which stores the DevID secrets, credentials and the entire credential chain up to the root certificate. It contains a strong RNG and Implements Asymmetric Algorithms (2048 bit RSA and/or 256 bit ECDSA) as well as a SHA-256 Hash Function.

A DevID is cryptographically bound to a device and supports authentication of the device's identity. An IDevID provided by the supplier of a device can be supplemented by LDevIDs facilitating enrollment (provisioning of authentication and authorization credentials) by local network administrators.

7.3.1. Use of DevIDs

EAP-TLS Authentication: Device authentication can be based on its DevID certificate.

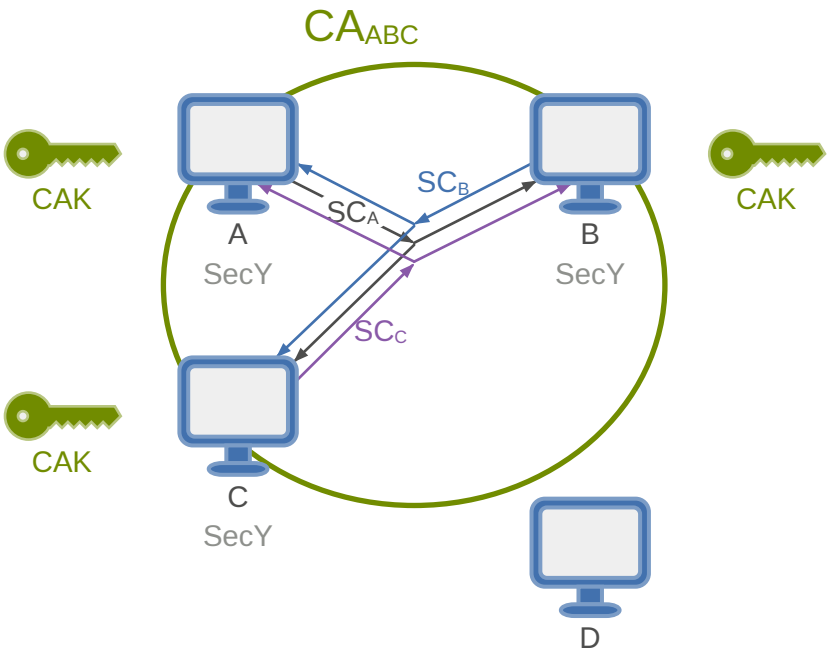
Consumer Devices: Similar but more secure than access control based on a MAC address list which can easily be spoofed. A switch, router or access point can allow access based on a registered commonName (CN), serialNumber (SN) or a subjectAltName contained in the DevID certificate.

Enterprise Devices: Similar to the consumer device use case but the DevID is usually registered with a central AAA server (white-listing).

DevID Module based on Trusted Platform Module (TPM): Each TPM has a unique non-erasable Endorsement Key (EK) to which DevID secrets and credentials can be bound.

7.4. MEDIA ACCCESS LAYER SECURITY (MACSEC) - IEEE 802.1AE

A network security standard that operates at the medium access control layer and defines connectionless data confidentiality and integrity for media access independent protocols.



D should not get any messages

CA: Connectivity Association, the “in group” which should be able to read the messages

SecY: MAC Security Entity

CAK: CA Key, shared between the MAC Security Entities in a CA

SC: Secure Channel, from one party to all the others. 3 entities = 3 channels.

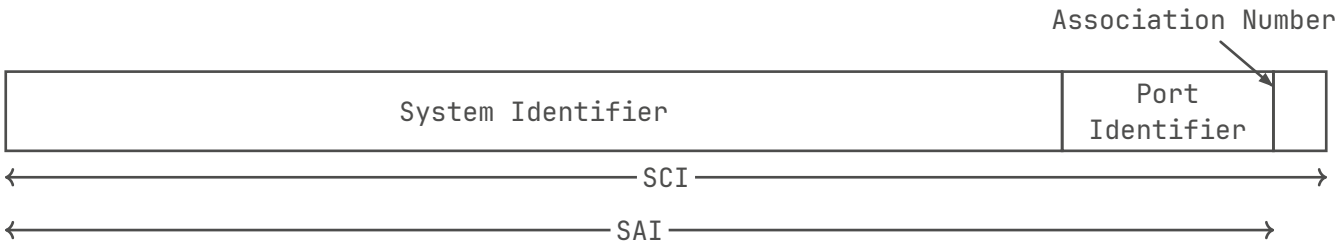
SA: Secure Association, like in IPSec IKE

Each SC comprises a succession of SAs each with a different SAK (SA Key) (for rekeying).

TODO:
two-station: eg client and switch, secure association

frame format: everything is encrypted

7.4.1. SC and SA Identifiers



SCI: Secure Channel Identifier

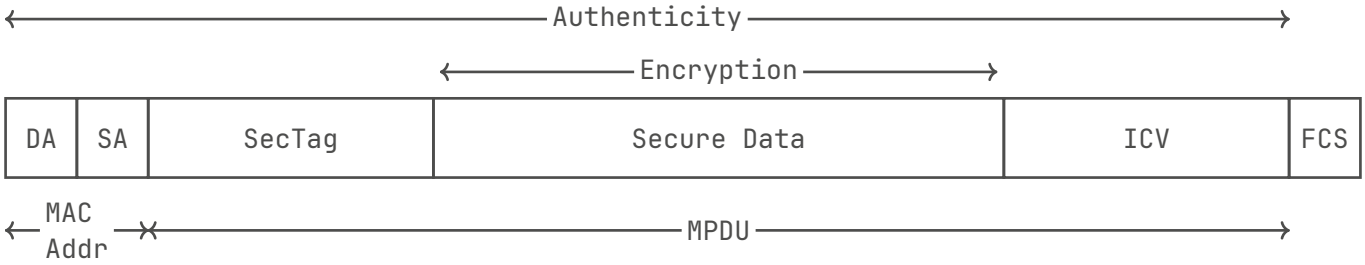
SAI: Secure Association Identifier

System Identifier: MAC address of sender

Port Identifier: Allows for multiple channels

The Association Number (2 bits) allows the overlapping rekeying of the Secure Association during which two different SAKs co-exist.

7.4.2. Frame Format

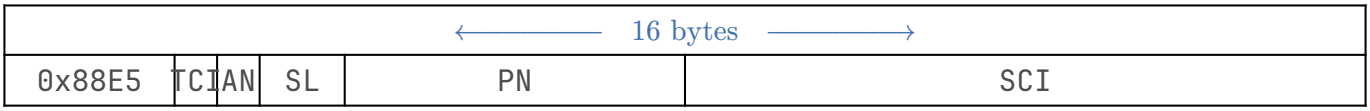


MSDU: MAC Service Data Unit

MPDU: MACsec Protocol Data Unit

ICV: Integrity Check Value

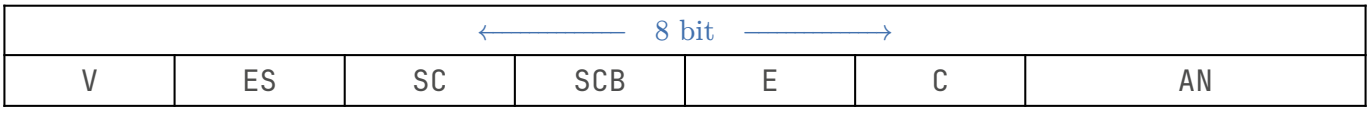
7.4.2.1. Security Tag (SecTag)



Field	*Definition*
0x88E5 (2 bytes)	MACsec Ethertype is 0x88E5
TCI (0.5 bytes)	TAG Control information
AN (0.5 bytes)	Association Number (2bits)
SL (1 bytes)	Short Length (6bits) - length of user data if < 48 octets, 0 otherwise
PN (4 bytes)	Packet Number - replay protection and IV for encryption
SCI (8 bytes)	Secure Channel Identifier (0 or 8 bytes) - identifies the secure channel.

In point-to-point links the SCI consists of the Source MAC Address and the fixed Port Identifier 00-01. Thus, no need to encode the SCI.

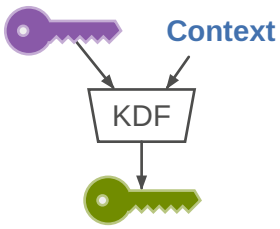
7.4.2.2. Tag Control Information Bits (TCI)



Field	*Definition*
V (1 bit)	version, currently 0
ES (1 bit)	If set (=1) means that the Source MAC Address is part of the SCI and the SCI does not need to be explicitly encoded
SC (1 bit)	If set an explicitly coded SCI is present in the SecTag
SCB (1 bit)	Single Copy Broadcast capability, if ES and SCB are set, then the implict SCI has a reserved Port Identifier of 00-00.
E (1 bit)	Encryption, if set means encryption enabled
C (1 bit)	Changed Text, if not set means the Secure Data is equal to User Data

7.4.3. MACsec Key Agreement (MKA) - IEEE 802.1X

KDF allow to generate keys for a context given a master key. It's deterministic and normally a PRF (HMAC or CMAC).



The MKA KDF is a PRF based on AES-CMAC with a 128 or 256 bit key.
Output ← KDF(Key, Label, Context, Length)



7.4.3.1. Connectivity Association Key (CAK)

CKN: (public) CA Key Name

CAK as a Pre-Shared-Key (PSK): Can be used either as a pairwise CAK or group CAK and is statically configured (admin must do it manually). CKN can be chosen arbitrarily with a size of 1..32 octets.

CAK via EAP: Can be used as a pairwise CAK. Dynamically derived CAK and CKN between two PAEs via EAP.

$$\text{CAK} \leftarrow \text{KDF}(\text{MSK}[0..15]/\text{MSK}[0..31], \text{"IEEE8021 EAP CAK"}, \quad \text{mac1}|\text{mac2}, 128/256)$$

$$\text{CKN} \leftarrow \text{KDF}(\text{MSK}[0..15]/\text{MSK}[0..31], \text{"IEEE8021 EAP CKN"}, \text{Session-ID}, \text{mac1}|\text{mac2}, 128/256)$$

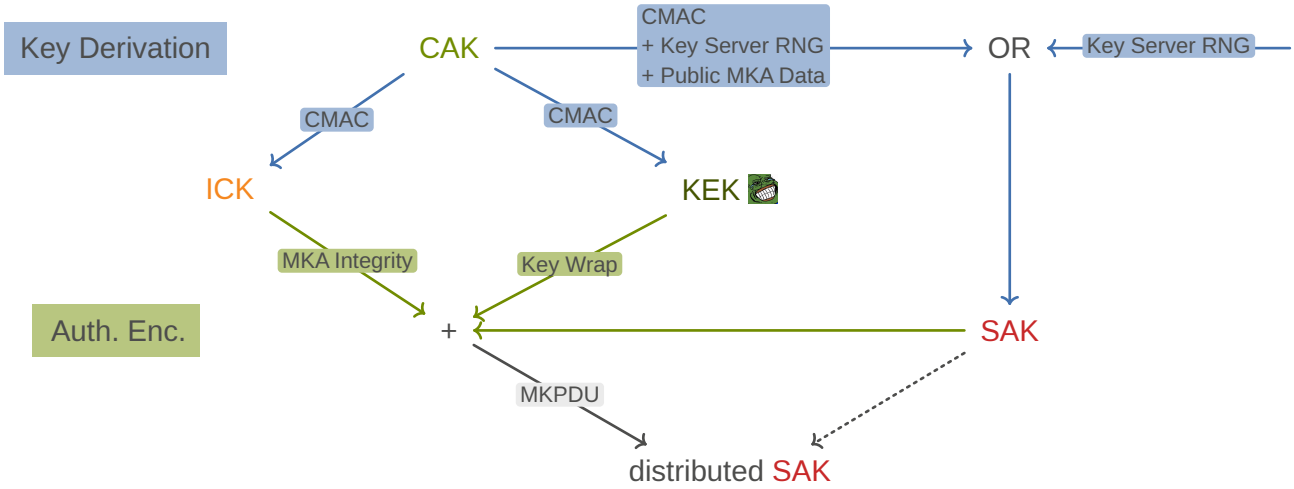
where mac1, mac2 are the MAC addresses of the PAEs and the Master Session Key (MSK) and Session-ID of the EAP method (EAP-TLS, EAP-PEAP, etc) is included.

7.4.3.2. Secure Association Key (SAK)

1) Members of CA have shared CAK
2) Key Server (KS) generates SAK From CAK via KDF or using fresh randomness
3) KS distributes SAK to other members of CA using MKPDU over EAPOL. MKPDU is an "ad-hoc" secure channel which requires keys for integrity (ICK) and encryption (KEK) which are generated from CAK via KDF.

KS can be e.g. the EAP Authenticator.

7.4.3.3. MKA random SAK distribution using CAK



CMAC: Is just a specific KDF in this case

Integrity key: MAC key

Wrapping key: encrypting it

MKPDU: MACsec Key Agreement Protocol Data Unit - carried via EAPOL

CAK: Connectivity Association Key - pairwise or group root key

ICK: ICV Key - used for MKPDU Data Integrity

KEK : Key Encrypting Key - used for AES Key Wrap in MKPDU

SAK: Secure Association Key

7.4.3.4. MKA Key Derivation Function for CAK

KS: Key Server - either elected or EAP Authenticator

MI: Member Identifier - all members of a CA

KN: Key Number - assigned by Key Server

$$\begin{aligned} \text{KEK} &\leftarrow \text{KDF}(\text{CAK}, \text{"IEEE8021 KEK"}, \text{CKN}[0..15], 128/256) \\ \text{ICK} &\leftarrow \text{KDF}(\text{CAK}, \text{"IEEE8021 ICK"}, \text{CKN}[0..15], 128/256) \\ \text{SAK} &\leftarrow \text{KDF}(\text{CAK}, \text{"IEEE8021 SAK"}, \text{KS-nonce} \mid \text{MI-value list} \mid \text{KN } 128/256) \end{aligned}$$

TODO:
W6 S47

7.5. TRUSTED NETWORK CONNECT

Host is given access to sanitized network only after successful Network Endpoint Assessment (NEA) Using Trusted Network Connect (TNC).

The Platform Trust Service (PTS), comprised of a TSS and TPM, verifies that the device is not “lying”. The root of trust is usually given by the TPM.

Integrity Measurement Verifiers (IMVs) are the pieces that gather information (e.g. open ports).

TODO:
W6 S56..

diagram:
– TNC TPM?